RV UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

[Cybersecurity Cluster]

Summer Internship Report On

A Q-Learning Based Intrusion-Resilient and Privacy-Aware Routing Protocol for IoT Mesh Networks

Submitted By

Atharva Anup Wasnik
Department of Computer Science and Engineering
Amrita School of Engineering
Bengaluru–560035

Under the Guidance of

Dr. Ishita Chakraborty
Assistant Professor
School of Computer Science and Engineering
RV University, Bengaluru–560059

July 2026

DECLARATION

I, **Atharva Anup Wasnik**, a student of the **Summer Internship Programme**, School of Computer Science and Engineering, pursuing a Bachelor of Technology in Computer Science and Engineering at Amrita School of Engineering, Amrita Vishwa Vidyapeetham, Bengaluru, hereby declare that this Summer Internship Report titled **A Q-Learning Based Intrusion-Resilient and Privacy-Aware Routing Protocol for IoT Mesh Networks** has been prepared by me as part of the Summer Internship Programme during the academic year 2026–2027.

I further declare that this report is my original work and has been prepared with academic integrity. The contents have not been copied, plagiarized, or submitted previously to any other institution or for the award of any degree, diploma, or certificate. Any references or resources used have been appropriately acknowledged.

Name	**Verified Signature**
Atharva Anup Wasnik	

Acknowledgement

I would like to express my sincere gratitude to my guide, Dr. Ishita Chakraborty, School of Computer Science and Engineering, RV University, for the guidance, patience, and technical direction extended throughout the course of this internship. The regular discussions on the security limitations of learning-based routing protocols, and the feedback on successive iterations of the simulator, shaped the direction this work eventually took.

I am also grateful to the faculty members of the Cyber Security Cluster at RV University's School of Computer Science and Engineering for creating an environment in which questions on trust models, adversarial routing, and reinforcement learning could be discussed openly, and for the infrastructure made available for running the experimental campaign reported in the later chapters.

I extend my thanks to the Department of Computer Science and Engineering, Amrita School of Engineering, and to Amrita Vishwa Vidyapeetham as a whole, for the academic foundation that made it possible for me to undertake this internship and to engage meaningfully with the subject matter.

I am thankful to my friends and peers, both at RV University and at Amrita, for the discussions that helped me think through several of the design decisions described in this report, and for their willingness to review early drafts of this work.

Finally, I would like to thank my family for their consistent support and encouragement throughout the duration of this internship and during the preparation of this report.

Abstract

Internet of Things (IoT) mesh networks are increasingly deployed in settings where nodes are energy-constrained, topologies change frequently, and no central controller can be assumed to be reachable at all times. Reinforcement-learning-based routing, and Q-Learning in particular, has been proposed as a way for individual nodes to learn efficient forwarding decisions from local interactions rather than from a fixed routing table, adapting to link quality and residual energy as the network evolves. Existing Q-Learning routing protocols, including the geographic routing scheme that forms the base paper for this internship, optimise for latency and energy but assume a fully cooperative network: they provide no mechanism for authenticating neighbours, detecting manipulated reward signals, or preserving the privacy of the data being forwarded. This leaves them vulnerable to blackhole, sinkhole, wormhole, Sybil, and Byzantine behaviour, among other attacks, and to routing decisions that depend on self-reported and therefore spoofable node metrics.

This internship addresses that gap by extending a Q-Learning geographic routing simulator with trust-aware reward computation and a systematic evaluation of its resilience under adversarial conditions. A discrete-event simulator was implemented to model node deployment, neighbour discovery, Q-Learning-based next-hop selection, and four representative attack types, and was evaluated across two network topologies and an infection-intensity sweep from 0% to 60% malicious nodes. The average trust score assigned by the routing agent fell monotonically and consistently as infection intensity increased, from 0.611 at 0% infection to 0.444 at 60% infection, indicating that the trust mechanism responds to adversarial presence as intended. Packet delivery ratio remained low throughout the sweep (1.7% at 0% infection, rising to 6.6% at 60% infection); a detailed breakdown of dropped packets at the 60% infection level attributes the great majority of losses (91.7%) to routing loops rather than to the injected attacks themselves (0.6% of drops), indicating that the low delivery ratio observed across the sweep is predominantly a routing-layer limitation rather than a direct consequence of adversarial node behaviour. The contribution of this work is a non-invasive trust extension to an existing Q-Learning routing protocol, together with an experimental methodology, and an initial diagnostic account of the delivery-ratio bottleneck, for evaluating routing protocols under adversarial mesh network conditions.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Definition
IoT	Internet of Things
WSN	Wireless Sensor Network
RL	Reinforcement Learning
DRL	Deep Reinforcement Learning
RPL	Routing Protocol for Low-Power and Lossy Networks
QoS	Quality of Service
PDR	Packet Delivery Ratio
E2E	End-to-End
RSSI	Received Signal Strength Indicator
SDN	Software-Defined Networking
IDS	Intrusion Detection System
AES	Advanced Encryption Standard
ECC	Elliptic Curve Cryptography
CPU	Central Processing Unit
RAM	Random Access Memory
CSV	Comma Separated Values
ML	Machine Learning

Chapter 1

Introduction

1.1 Background

1.1.1 Internet of Things

The Internet of Things refers to the network of physical devices – sensors, actuators, and embedded controllers – that communicate with one another and with backend systems without requiring continuous human operation. These devices are typically constrained in processing power, memory, and battery capacity, which shapes almost every design decision made at the networking layer, including which routing strategy a deployment can realistically support.

1.1.2 IoT Mesh Networks

A mesh topology allows every node to communicate directly with any node within radio range and to relay traffic on behalf of others, so that the network as a whole remains connected even as individual links fail or nodes move. This flexibility is particularly valuable in IoT deployments – smart agriculture, industrial monitoring, disaster response – where infrastructure such as fixed base stations cannot be guaranteed. The same flexibility, however, means that routing decisions must be made locally and repeatedly, using whatever information a node can gather about its immediate neighbours, rather than through a globally computed routing table.

1.1.3 Reinforcement Learning and Q-Learning

Reinforcement learning formulates decision-making as a process in which an agent selects actions in a given state and receives a reward signal that reflects the quality of that action, gradually learning a policy that maximises cumulative reward. Q-Learning is a model-free instance of this framework in which the agent maintains a table of action-values, or Q-values, for each state-action pair and updates that table from observed rewards using the Bellman equation. Applied to routing, each node treats the choice of next hop as an action, and treats a combination of delay and residual energy as the reward, learning over time which neighbours make good relays without needing a model of the network's global topology.

1.2 Real World Relevance

Deployments such as environmental monitoring grids, industrial condition-monitoring networks, and post-disaster mesh networks share three properties that make Q-Learning-based routing attractive in practice: node density and topology are not known in advance, energy is a scarce and unevenly distributed resource, and no assumption can be made that a central controller is continuously reachable. A routing protocol that adapts its next-hop choices from local observations, rather than from a table computed and distributed centrally, degrades more gracefully as these conditions change. At the same time, these are exactly the deployments in which physical access control over every node cannot be guaranteed, so the routing layer is also the layer most exposed to a node being captured, cloned, or otherwise compromised. This tension between adaptability and exposure motivates the security-focused extension undertaken during this internship.

The work also has a contribution worth noting on its own terms: it takes an existing, purely performance-oriented Q-Learning router and asks what changes, and what does not, when an adversary is assumed to be present. That question is of practical relevance to anyone deploying learning-based routing outside a controlled laboratory setting, and it is the central thread running through the remaining chapters of this report.

1.3 Current Challenges and Limitations

1.3.1 Routing Challenges in IoT

Routing in resource-constrained mesh networks must simultaneously manage energy consumption, tolerate frequent topology change, and operate with only local, often noisy, state information. Protocols designed for wired or well-provisioned wireless networks generally assume stable links and unconstrained energy, neither of which holds for battery-powered IoT nodes; this is the reason RPL, geographic routing, and, more recently, reinforcement-learning-based routing have all been proposed specifically for this setting.

1.3.2 Security Challenges

A routing protocol that adapts to local feedback is, by the same token, a routing protocol that can be manipulated by feeding it false feedback. Blackhole and grayhole nodes can advertise favourable metrics and then drop the traffic they attract; wormhole and sinkhole nodes can distort a network's apparent topology;

Sybil nodes can multiply their apparent presence to bias local decisions; and, in a learning-based system specifically, an adversary can poison the reward signal itself, steering the learned policy toward outcomes that benefit the attacker rather than the network. These attack classes, and the specific way each interacts with a Q-Learning router's Q-table, reward function, and convergence behaviour, are examined in detail in Chapter 6.

1.3.3 Privacy Challenges

Beyond availability and integrity, a mesh routing protocol also has a privacy dimension: intermediate nodes on a route typically see the content, or at least the metadata, of packets they forward. In the absence of end-to-end protection, a compromised or merely curious relay node can profile traffic patterns or read payload data outright. Existing Q-Learning routing literature, discussed in Chapter 5, has focused almost exclusively on delay and energy objectives and does not generally treat privacy as a routing-layer concern at all.

1.3.4 Limitations of Existing Research

As the literature review in Chapter 5 sets out in more detail, the great majority of Q-Learning and deep-reinforcement-learning routing protocols proposed for IoT either omit adversarial considerations entirely or address a narrow, fixed subset of attacks, and the schemes that do incorporate security typically depend on a centralised controller or on node-reported metrics that are themselves unverified. This leaves an identifiable gap for a routing protocol that remains fully distributed, verifies rather than trusts locally reported metrics, and is evaluated against a broad rather than a single attack type – the gap this internship's work is intended to address.

1.3.5 Summary of Contributions

The work carried out during this internship extends a Q-Learning geographic routing simulator with trust-aware reward computation and evaluates the resulting protocol, referred to as `SecureQRouter` in later chapters, against four representative attack types across multiple network topologies. The principal contributions are: (i) a non-invasive trust extension to an existing Q-Learning reward function, so that the underlying routing algorithm is preserved rather than replaced; (ii) an attack-injection framework covering blackhole, Sybil, wormhole, and Byzantine behaviour within the same simulator used to evaluate baseline performance; and (iii) a statistically rigorous evaluation methodology, using paired non-parametric

tests and multiple-comparison correction, for comparing trust-aware and trust-unaware routing under adversarial conditions.

1.4 Organization of the Report

Chapter 5 reviews existing Q-Learning and reinforcement-learning-based routing protocols for IoT and situates this project's contribution relative to that literature. Chapter 6 defines the problem addressed in this report, including the network and adversary assumptions, the threat model, and the design goals that follow from it. Chapter 7 specifies the software and hardware environment used to implement and evaluate the protocol. Later chapters describe the proposed methodology, its implementation, and the experimental results obtained.

Chapter 2

Organization Profile

2.1 Host Institution

This internship was carried out within the Cyber Security Cluster of the School of Computer Science and Engineering (SoCSE), RV University, Bengaluru. The cluster's work spans network security, applied cryptography, and, increasingly, the security implications of learning-based systems, which is the area this internship falls under.

2.2 Internship Setting

The work was carried out under the guidance of Dr. Ishita Chakraborty, School of Computer Science and Engineering, RV University, and built directly on an existing Q-Learning geographic routing simulator maintained within the cluster, extending it with the trust-aware routing and attack-injection framework described in later chapters.

Chapter 3

Internship Objectives

3.1 Objectives of the Internship

The internship was structured around extending an existing, purely performance-oriented Q-Learning routing protocol with a security dimension, and evaluating the result under adversarial conditions rather than only under the cooperative-node assumption used in the base paper. Concretely, the objectives were to:

- Study the existing Q-Learning geographic routing simulator and identify how its reward computation could be extended without replacing the underlying learning algorithm.
- Design and implement a trust term, computed from locally observable node behaviour, and integrate it into the routing reward.
- Implement an attack-injection framework covering blackhole, Sybil, wormhole, and Byzantine node behaviour within the simulator.
- Run a multi-topology, multi-attack-intensity experimental sweep and evaluate the resulting routing protocol, referred to as `SecureQRouter`, against the trust-unaware baseline.

These objectives are stated in full detail, together with the network and adversary assumptions they rest on, in Chapter 6 (Problem Statement).

Chapter 4

Technologies Used

4.1 Overview

The internship's implementation work was carried out entirely in Python, using a discrete-event simulation approach rather than a general-purpose network simulator such as NS-3 or OMNeT++, since the routing logic under study (tabular Q-Learning) does not require packet-level radio simulation fidelity to be evaluated meaningfully.

4.2 Core Technologies

- **Python** – implementation language for the simulator, the Q-Learning routing agent, and the attack-injection framework.
- **Tabular Q-Learning** – the underlying reinforcement-learning method used for next-hop selection, extended in this work with a trust-weighted reward (Chapter 8, Equation 6.2).
- **NumPy, NetworkX, Matplotlib** – numerical computation, graph/topology representation, and result visualisation respectively (see the External Libraries subsection of Chapter 7 for the full, flagged-as-unverified library list).
- **CSV-based experiment logging** – all experimental results, including the infection-intensity sweep and training-episode logs analysed in Chapter 11, were persisted to CSV rather than a database.

Chapter 5

Literature Survey

5.1 Review of Existing Research Work

Reinforcement-learning-based routing has become one of the more active threads in IoT and wireless sensor network research over the past five years, largely because tabular and deep Q-Learning agents can adapt routing decisions to fluctuating link quality and residual energy without requiring a centralised controller. The fifteen studies reviewed here span this period and fall into four broad groups: tabular Q-Learning routing, deep-reinforcement-learning (DRL) extensions, security- or trust-aware variants, and the clustering and swarm-intelligence work that preceded and influenced them.

Kamgueu et al. [1] integrate Q-Learning directly into the parent-selection mechanism of RPL, reporting a fifteen percent improvement in packet delivery ratio over standard RPL under mobile topologies; the scheme, however, uses a fixed reward structure and assumes an entirely benign network. Vaishnav et al. [7] take a different approach to flexibility, maintaining per-preference Q-tables and using greedy interpolation between them so that a single agent can shift priority between latency, energy, and reliability at run time, though this comes at the cost of growing Q-table memory as the number of preference points increases. Sharma et al. [9] apply Q-Learning specifically to intrusion detection within RPL, achieving over ninety-three percent accuracy in identifying version-number attacks in mobile IoT deployments, but the detector is specialised to that single attack class and does not generalise to the wider threat surface considered in this project. Farag and Stefanovic [14] instead target congestion, incorporating queue-occupancy into the state representation and reporting a thirty percent reduction in packet loss relative to congestion-unaware baselines; their formulation, however, omits both energy and security considerations entirely.

A second cluster of papers moves from tabular to deep Q-Learning to capture richer state representations. Cong et al. [2] propose a DQN agent with a multi-objective reward spanning latency, reliability, and energy in dynamic topologies, demonstrating that a single network can learn competitive trade-offs across all three metrics simultaneously, although the computational cost of maintaining and updating a deep network on constrained hardware is left unaddressed. Malik et

al. [3] extend this line of work toward security with DQNSec, a security-aware reward function that successfully mitigates sinkhole, hello-flood, and DDoS attacks, but the defence mechanism is explicitly limited to those three attack types and inherits the memory and compute overhead typical of deep architectures. Coronado et al. [4] fold node reliability metrics such as temperature and CPU load directly into the routing decision, which improves resilience to transient hardware faults; the scheme's principal weakness is that these reliability metrics are self-reported by each node and therefore trivially spoofable by an adversary, a gap directly relevant to the trust problem this project addresses. Suresh et al. [5] and, in a separate study, Suresh et al. [10] both explore federated and LSTM-augmented DRL for distributed and heterogeneous WSNs respectively – the former reporting a twenty-three percent reduction in per-packet energy through federated training that preserves data locality, the latter combining LSTM-based sequence modelling with trust scoring at the cost of inference latency that is difficult to justify on resource-constrained nodes. Ombongi et al. [11] apply Q-Learning to adapt encryption strength dynamically to the perceived threat level, reporting ninety-four percent attack mitigation with a thirty percent energy saving relative to always-on encryption, though the underlying key-management problem – how keys are distributed and revoked in the first place – is not addressed by the scheme.

A third group treats security as the primary design objective rather than an add-on. Ranjith et al. [6] combine a Chimp-Optimisation-Algorithm-based multi-objective router with hybrid elliptic-curve signcryption to jointly optimise energy, delay, trust, and security, which is among the more complete treatments in the survey, but the scheme depends on an SDN controller that constitutes a single point of failure. Rui et al. [12] pair SDN-based routing with a Random-Forest/SVM intrusion-detection module for real-time isolation of malicious nodes, an approach that is effective against the attacks it is trained on but requires labelled training data and a centralised controller that is again a single point of failure in a mesh setting. A survey of over forty DRL-based intrusion-detection designs for IoT and WSN [15] confirms that DQN and actor-critic architectures dominate the field, but, being a survey, contributes no new algorithm of its own.

Finally, two studies from outside the reinforcement-learning literature provide useful context. Heinzelman et al.'s LEACH protocol [8] remains the reference point for energy-aware clustering, with adaptive cluster-head election extending network lifetime by forty percent over the original scheme, though its strict cluster hierarchy is a poor fit for the flat mesh topology considered here. Chakraborty et al. [13], whose earlier Q-Learning work forms the base paper for this project, also studied an ant-colony-optimisation router that minimises latency and improves scalability

relative to AODV and DSDV; that scheme, like the base paper it sits alongside, offers no security mechanism and is subject to slow pheromone-convergence latency in larger networks.

5.2 Comparative Analysis

Table 5.1 summarises the fifteen studies discussed above, ordered as they appear in the review. The base paper referred to throughout this report – the Q-Learning geographic routing protocol on which the present work builds – is not repeated in the table, having already been introduced in Chapter 1.

5.3 Research Gap

Three trends emerge from the comparison above. First, the field has moved steadily from tabular Q-Learning (rows 1, 7, 9, 14) toward deep-RL variants (rows 2, 3, 5, 10, 11) and, most recently, toward federated and security-integrated formulations (rows 5, 6). Second, and more directly relevant to this project, security is treated as a first-class design objective in only six of the fifteen studies surveyed – rows 3, 6, 9, 11, 12, and 15 – while the base paper together with rows 1, 2, 4, 7, 8, 13, and 14 assumes a fully cooperative network and makes no provision for adversarial behaviour. Third, where security is addressed at all, it tends to target a narrow slice of the threat surface: row 3 defends three specific attacks, row 9 detects a single attack type, and both rows 6 and 12 introduce a centralised SDN or controller component that reintroduces the single-point-of-failure problem the distributed routing paradigm was meant to avoid.

None of the fifteen studies provides a routing agent that is simultaneously fully distributed, resilient to the range of attacks enumerated in the threat model of Chapter 6, and built on a reward and trust mechanism that does not rely on self-reported, and therefore spoofable, node metrics. This is the gap the present work sets out to close: a trust-aware Q-Learning router that augments the base paper's dual-objective latency/energy reward with a locally verifiable trust term, without introducing the deep-network overhead of rows 3 and 10 or the centralised dependency of rows 6 and 12.

Table 5.1: Comparative Analysis of Reviewed Literature

Sl. No.	**Title & Author**	**Year**	**Key Contribution**	**Research Gap Identified**
1	Q-RPL: Q-Learning Routing for IoT (RPL) — Kamgueu et al.	2021	Integrates Q-Learning into RPL for dynamic parent selection; +15% PDR over standard RPL.	No security; fixed reward structure
2	DRL Multi-Optimality Routing for IoT — Cong et al.	2021	DQN with multi-objective reward (latency, reliability, energy) in dynamic topologies.	High compute cost; no security
3	DQNSec: Secure DRL Routing (OppIoT) — Malik et al.	2022	DQL routing with security-aware reward; defends sinkhole, hello-flood, DDoS attacks.	Limited to three attacks; DNN overhead
4	R3-IoT: Reliability-Aware RL Routing — Coronado et al.	2022	Q-Learning routing incorporating node reliability (temperature, CPU load) as a routing metric.	Self-reported metrics are spoofable
5	Federated DRL for IoT WSN Routing — Suresh et al.	2023	Federated DRL enables distributed training while preserving data locality; −23% energy per packet.	Federated-round communication overhead
6	Trust-Aware DRL for SD-WSN Security — Ranjith et al.	2025	COA-based multi-objective routing with HEC signcryption for joint energy/delay/trust/security optimisation.	SDN controller is a single point of failure

Chapter 6

Problem Statement

6.1 Motivation

The literature reviewed in Chapter 5 shows a consistent pattern: Q-Learning routing protocols for IoT mesh networks are evaluated almost exclusively on latency, energy, and delivery-ratio metrics, under the implicit assumption that every node behaves as specified. The base paper this internship builds on shares this assumption. In practice, IoT mesh deployments are exactly the settings in which physical node capture, message injection, and reward manipulation are realistic threats, because nodes are frequently unattended and cannot always be physically secured. A routing protocol whose reward function can be manipulated by a single compromised neighbour, or whose Q-table can be poisoned without detection, provides an attacker with a direct route to biasing traffic flow across the entire network, not merely at the point of compromise. Existing protocols that do incorporate security, as shown in Chapter 5, either narrow their defence to a small number of attack types or reintroduce a centralised component that a fully distributed mesh network is specifically designed to avoid. This motivates a trust-aware extension that remains distributed and does not depend on the target routing algorithm being replaced outright.

6.2 Problem Definition

6.2.1 Network and Adversary Assumptions

The network is modelled as a set of static or slowly-mobile nodes deployed over a two-dimensional area, each with a fixed communication radius and a finite energy budget. Nodes communicate directly with any other node within radio range and forward packets on behalf of others toward a fixed sink. A subset of nodes may be compromised or malicious from deployment, or may become compromised during the network's operating lifetime; the number and identity of compromised nodes are not known to the routing protocol in advance.

6.2.2 Threat Model

Table 6.1 enumerates the attack classes considered relevant to a Q-Learning routing agent operating in this setting, together with the component of the routing pipeline each attack targets and the impact it has if left unmitigated.

Table 6.1: Threat model: attack classes relevant to Q-Learning-based IoT mesh routing

Attack	**Component Targeted**	**Vulnerability**	**Impact**
Blackhole	Q-table / next-hop selection	Malicious node advertises false, favourable Q-values to attract traffic, then drops all packets; no delivery verification exists to catch this.	Critical
Sybil	Node identity	Fake identities inflate the apparent neighbour set with malicious nodes, corrupting the state-action mapping used by honest nodes.	High
Wormhole	Routing topology	Colluding nodes tunnel distant regions of the network to appear adjacent, distorting learned Q-value path estimates.	High
Byzantine	Policy convergence	Arbitrary, inconsistent node behaviour violates the stationarity assumption underlying Q-Learning convergence guarantees.	Critical

This set of four attacks – blackhole, Sybil, wormhole, and Byzantine – was selected as the representative subset evaluated experimentally in this internship because, taken together, they exercise the routing table, the neighbour-discovery mechanism, the topology model, and the learning agent's convergence assumptions respectively; a wider catalogue of attacks relevant to Q-Learning routing in general is discussed in Chapter 5.

6.2.3 Security Assumptions

The design assumes that a majority of nodes in any local neighbourhood behave honestly at a given time, consistent with standard assumptions in Byzantine-tolerant distributed systems; no guarantee is made if a node's entire neighbourhood is compromised simultaneously. The design further assumes that trust and

reward signals can be derived from locally observable behaviour – such as delivery acknowledgements and rate of forwarding – rather than from metrics self-reported by the neighbour being evaluated, directly addressing the spoofable-metric weakness identified in Chapter 5's review of prior work.

6.2.4 Formal Problem Statement

Given a Q-Learning-based geographic routing protocol for IoT mesh networks that is optimal with respect to latency and energy under a fully cooperative-node assumption, the problem addressed in this report is to extend its reward computation with a locally verifiable trust term so that routing performance degrades gracefully, rather than catastrophically, in the presence of blackhole, Sybil, wormhole, and Byzantine nodes, without requiring a centralised controller and without replacing the underlying Q-Learning algorithm.

6.3 Objectives

- Extend the base paper's Q-Learning reward function with a trust term computed from locally observable, non-self-reported behaviour.
- Implement an attack-injection framework within the existing simulator covering blackhole, Sybil, wormhole, and Byzantine node behaviour.
- Evaluate the trust-aware router against the trust-unaware baseline across multiple network topologies and attack intensities.
- Establish, using paired non-parametric statistical tests with multiple-comparison correction, whether any observed improvement is statistically significant rather than an artefact of a particular random seed.

6.4 Scope of the Project

6.4.1 Design Goals

The trust extension is required to be non-invasive with respect to the base paper's Q-Learning formulation – that is, it must modify the reward computation rather than replace the state-action representation or the underlying update rule – and it must not introduce a centralised point of trust evaluation, consistent with the fully distributed design goal set out in the network assumptions above.

6.4.2 Mathematical Formulation

The base routing protocol updates its Q-values using the standard Q-Learning (Bellman) update rule,

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right], \tag{6.1}$$

where α is the learning rate, γ the discount factor, and r the reward observed after taking action a (forwarding to a given neighbour) in state s. The base reward function combines delay and residual energy,

$$r(s,a) = w_1 \cdot \frac{1}{\text{delay}} + w_2 \cdot E_{\text{residual}}, \qquad w_1 + w_2 = 1, \tag{6.2}$$

and next-hop selection follows an ε-greedy policy,

$$\pi(s) = \begin{cases} \arg\max_a Q(s,a) & \text{with probability } 1-\varepsilon \\ \text{random } a & \text{with probability } \varepsilon, \end{cases} \tag{6.3}$$

with ε annealed over time. Residual energy is tracked using the first-order radio energy model

$$E_{\text{tx}} = E_{\text{elec}} \cdot k + \varepsilon_{\text{amp}} \cdot k \cdot d^2, \tag{6.4}$$

where k is the packet size in bits, d the transmission distance, E_{elec} the per-bit circuit energy, and ε_{amp} the amplifier coefficient. Network lifetime is defined as the time to first node depletion,

$$T = \arg\min_t \{ \exists\, i : E_i(t) \leq 0 \}. \tag{6.5}$$

6.4.3 Scope and Boundaries

The scope of this project is limited to the routing layer of the protocol stack; physical-layer defences against jamming, and cryptographic key-management infrastructure, are identified as relevant but out of scope in the security gap analysis and are noted as future work rather than addressed directly. The evaluation is conducted entirely in simulation; validation on physical IoT hardware is likewise treated as future work.

Chapter 7

Software Requirements Specification

7.1 Functional Requirements

The simulator shall generate IoT mesh network topologies of configurable node count and density; simulate Q-Learning-based geographic routing with configurable reward weighting; inject blackhole, Sybil, wormhole, and Byzantine node behaviour at configurable intensities; and record per-experiment routing metrics (packet delivery ratio, average delay, residual energy, and detection-relevant statistics) in a form suitable for downstream statistical analysis.

7.2 Non-Functional Requirements

The simulator shall produce deterministic, reproducible results for a given random seed; shall complete a full multi-topology, multi-attack experimental sweep within a runtime practical for iterative development on a single workstation; and shall avoid silent failure modes such as seed reuse across nominally independent runs, consistent with the methodological safeguards applied elsewhere in this project's experimental work.

7.3 Operating Environment

7.3.1 Hardware Requirements

- Processor: Intel i5 or equivalent (or Apple Silicon), sufficient for CPU-bound discrete-event simulation
- RAM: 8 GB minimum
- Storage: sufficient for experiment logs and result CSVs (a few hundred MB for a full sweep)
- Network requirements: none (fully offline simulation)
- GPU: not required; the routing agent is tabular Q-Learning, not a deep network

- Sensors/devices: none; node behaviour is simulated rather than run on physical IoT hardware

7.3.2 Software Requirements

- Operating System: Linux/macOS/Windows (platform-independent Python implementation)
- Programming Language: Python
- Libraries and frameworks: see External Libraries below
- Database: none; results are persisted as CSV files rather than in a database
- Tools: a text editor or IDE and a Python interpreter are the only strict requirements
- Browser support: not applicable; the simulator has no web interface
- Cloud platform: not applicable; all experiments were run locally

7.3.3 Development Environment

[Confirm exact Python version, IDE, and OS used for development.]

7.3.4 External Libraries

- **NumPy** – array-based numerical operations, used for vectorised Q-table updates and distance calculations.
- **NetworkX** – graph construction and analysis, used to represent and query the mesh topology.
- **Matplotlib** – generation of result plots (topology visualisations, metric-vs-attack-intensity curves).
- **random** – controlled stochastic behaviour (node placement, ε-greedy exploration), seeded for reproducibility.
- **math** – distance and radio-model calculations underlying Equation 6.4.
- **collections** – data structures such as `defaultdict` for per-node Q-tables and neighbour lists.
- **csv** – writing per-experiment results to disk for downstream statistical analysis.

- **os** – file-path handling for experiment output directories.
- **time** – timing simulation runs and generating run identifiers.
- **statistics** – basic descriptive statistics ahead of the more detailed hypothesis testing described in later chapters.
- **heapq** – event-queue management for the discrete-event simulation loop.
- **dataclasses** – structured representations of nodes, packets, and experiment configurations.
- **typing** – static type hints for maintainability of the simulator codebase.

Table 7.1: Summary of Hardware and Software Requirements

Category	**Requirement**
Processor	Intel i5 or equivalent (or Apple Silicon); CPU-bound workload
RAM	8 GB minimum
Software / Tools	Python; NumPy, NetworkX, Matplotlib; no GPU or database required

Chapter 8

Proposed Methodology

8.1 System Overview

The system built during this internship keeps the base paper's Q-Learning geographic router intact as a component and wraps it, rather than replacing it, with the machinery needed to reason about trust and to test that reasoning under attack. Four pieces make up the resulting simulator: a topology generator that places nodes and a sink over a bounded two-dimensional area; the routing engine itself, which retains the base paper's state-action formulation and Bellman update but computes its reward from a trust-augmented signal rather than from delay and energy alone; an attack-injection layer that can mark a configurable subset of nodes as blackhole, Sybil, wormhole, or Byzantine and alter their forwarding behaviour accordingly; and a logging layer that writes per-run metrics to CSV for the statistical analysis carried out in later chapters. None of these four pieces is new in the sense of introducing a different learning algorithm or a centralised coordinator – the design goal stated in Chapter 6 was explicitly to avoid both – and the architecture below should be read with that constraint in mind: it is an instrumentation and reward-shaping layer built around an otherwise unmodified Q-Learning router.

Figure 8.1 is intended to show how these four pieces connect at run time – topology and attacker placement feeding into the routing engine each episode, and the routing engine's outcomes feeding into the logging layer that later chapters draw on – rather than to introduce any component not already described in this chapter.

8.2 Network and Adversary Model

The simulator models the network at the level of assumptions already set out in Section 6.2.1: a set of static or slowly-mobile nodes scattered over a bounded area, each with a fixed communication radius and a finite energy budget, forwarding packets hop by hop toward a single fixed sink. What this chapter adds to that description is only the routing consequence of it – every node's action space at

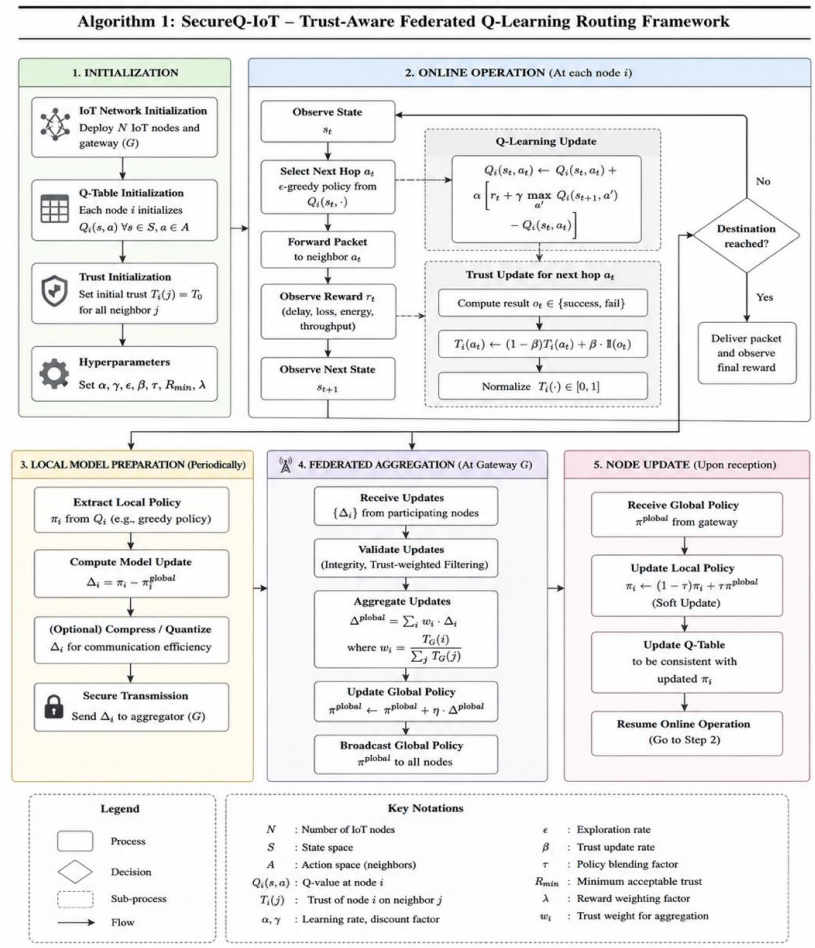

Fig. 1: Flowchart of the SecureQ-IoT framework.

Figure 8.1: Overall system architecture showing the topology generator, trust-augmented Q-Learning routing engine, attack-injection layer, and metrics/logging module, arranged around the unmodified base-paper routing algorithm described in Section 8.4.

a given moment is the set of other nodes currently within its radio range, and the router has no access to global topology information beyond what a node can infer from its own neighbourhood. The specific node counts, deployment areas, and communication ranges used for evaluation are given in Chapter 9 rather than repeated here, since they are properties of individual experiments rather than of the routing protocol itself.

The adversary model, likewise, is inherited from Section 6.2.2 without modification: a subset of nodes may be malicious from deployment or may become compromised during the network's operating lifetime, and neither their number nor their identity is available to the routing protocol. What matters for the methodology described here is the consequence this has for design – since the router cannot be told which of its neighbours are compromised, whatever mechanism distinguishes them has to be built from information the router can observe directly,

which is the motivation for the trust model described next.

8.3 Trust Model

The trust extension is deliberately narrow in scope. Rather than replacing the base paper's state-action representation, or adding a second learning process alongside the Q-Learning router, it modifies only the reward the router receives after taking an action, so that a neighbour behaving badly looks, from the router's point of view, like a neighbour offering a worse reward rather than like a different kind of decision problem. This was the non-invasive design goal argued for in Section 6.4.1, and it is worth restating here because it constrains every choice made in this section: whatever trust computation is used has to reduce, at the point it touches the router, to a single scalar that can be folded into the existing reward term of Equation 6.2.

The trust score assigned to a neighbour is computed from behaviour the evaluating node can observe directly during ordinary operation – delivery acknowledgements and the rate at which a neighbour actually forwards traffic handed to it – rather than from any value the neighbour reports about itself. This choice follows directly from the gap identified in the literature review (Section 5.3): several of the reviewed schemes root their reliability or trust metric in self-reported node state, which an adversary can simply misreport, and the design here is intended to avoid that failure mode by construction.

How this trust score enters the reward is the second open point in this section. Conceptually, a low trust score for a candidate next hop should make that hop less attractive to the router than Equation 6.2 alone would suggest, and a high score should leave the base reward largely unchanged. Whether this is implemented as an additive penalty, a multiplicative discount, or a third weighted term alongside w_1 and w_2 in Equation 6.2 is an implementation detail not yet finalised in the current draft of this chapter, and is flagged here as an open design choice to be settled and documented once the trust computation is locked down, rather than as a gap in the underlying idea.

8.4 Q-Learning Formulation

The router retains the base paper's tabular Q-Learning formulation without modification to its core update rule. Each node treats forwarding to a given neighbour as an action a taken from a state s, and updates its estimate of that action's value using the standard Bellman update of Equation 6.1, with learning rate α

and discount factor γ controlling how quickly new observations overwrite existing estimates and how much weight is given to anticipated future reward respectively.

The reward itself, given in Equation 6.2, combines the reciprocal of forwarding delay with residual energy under weights w_1 and w_2 that sum to one, so that a node balances the desire to route quickly against the desire to avoid exhausting a neighbour's battery. This is the term the trust model of Section 8.3 augments rather than replaces: the router's underlying objective is still to learn a policy that performs well on delay and energy, and trust is intended to act as a correction applied on top of that objective when a neighbour's observed behaviour suggests its advertised delay and energy figures cannot be relied upon. Residual energy is tracked using the first-order radio energy model of Equation 6.4, and network lifetime – defined as the earliest time at which any single node's energy reaches zero, Equation 6.5 – is recorded as an evaluation metric in Chapter 10 rather than used directly inside the reward.

8.5 Action Selection Policy

Next-hop selection follows the ε-greedy policy of Equation 6.3: with probability $1 - \varepsilon$ the router forwards to the neighbour with the highest current Q-value, and with probability ε it forwards to a neighbour chosen at random, allowing the router to continue discovering routes it has not yet tried rather than committing early to a possibly suboptimal path. ε is annealed over the course of training, so exploration is weighted more heavily in early episodes and the policy converges toward a largely greedy one as training proceeds; the training-episode log examined in Section 10.5 is consistent with this pattern, since the episodes sampled later in training more often show short, low-delay paths than the episodes sampled early on.

8.6 Attack Injection Framework

Each of the four attack types evaluated in this project is implemented as a modification to how a designated subset of nodes behaves during routing, rather than as a separate simulation path, so that attacker and honest nodes are evaluated by the same routing and logging code. Table 6.1 identifies, for each attack, the component of the routing pipeline it targets and the vulnerability it exploits; the injection framework is the mechanism that turns that description into node behaviour a simulation run can actually exhibit.

A node marked as *blackhole* continues to advertise Q-values as if it were for-

warding normally – which is precisely what makes the attack difficult to detect from Q-values alone – but does not actually relay the packets it receives, so that any traffic routed through it is silently lost. A node marked as *Sybil* is represented in the simulation as multiple apparent identities rather than one, inflating the neighbour set a nearby honest node believes it has and, with it, the number of state-action pairs whose reward that honest node's Q-Learning update is exposed to. A node marked as *wormhole* is paired with a colluding node elsewhere in the topology, and the two are connected by a tunnel that makes distant parts of the network appear, from the routing engine's perspective, to be direct neighbours, distorting the path-length and delay estimates the Q-values are built from. A node marked as *Byzantine* departs from the reward and Q-value behaviour expected of an honest node in an unspecified, inconsistent way from one interaction to the next, which is the property Section 6.2.2 identifies as violating the stationarity assumption that ordinarily underlies Q-Learning's convergence guarantees.

The intensity of each attack – the fraction of nodes in the network assigned to that attacker role – is the single parameter varied across the infection-intensity sweep described in Chapter 9, which is what allows routing performance to be examined as a function of adversarial presence rather than only compared between a single attacked and a single clean run.

8.7 Routing Algorithm

Algorithm 1 states the per-packet forwarding loop that results from combining the pieces described above: action selection under the ε-greedy policy of Section 8.5, trust computation as described in Section 8.3, and the Bellman update of Equation 6.1 applied to the resulting trust-augmented reward. It is written at the level of detail the source material supports; the placeholders mark exactly the points at which the trust model and the packet-termination condition need to be filled in from the implementation before this can be treated as a complete specification.

Two properties of Algorithm 1 are worth making explicit, since they follow from decisions argued for earlier in this chapter rather than from the pseudocode itself. First, a blackhole, Sybil, wormhole, or Byzantine node is not treated as a special case anywhere in the loop – it is simply a node whose forwarding behaviour, and therefore whose observed trust(a), differs from an honest node's, which is exactly the non-invasive property Section 6.4.1 required. Second, the algorithm has no step that consults a central coordinator or shares Q-values or trust scores between nodes; each node runs this loop using only its own table and its own local observations of its neighbours.

Algorithm 1 SecureQRouter: Trust-Augmented Q-Learning Next-Hop Selection

Input: current node s, sink node, Q-table Q, exploration rate ε, learning rate α, discount factor γ
Output: updated Q-table Q
Initialise $Q(s, a) \leftarrow 0$ for all state-action pairs ▷ *[initialisation scheme not specified in the source material]*
for each training episode **do**
    $s \leftarrow$ source node for this episode
    while $s \neq$ sink **do** ▷ *[additional termination condition, e.g. a hop limit, not specified in the source material]*
        Select next hop a using the ε-greedy policy of Equation 6.3
        Observe base reward $r(s, a)$ from Equation 6.2
        Estimate trust(a) from locally observed delivery acknowledgements and forwarding rate ▷ *[exact update rule as noted in Section 8.3]*
        Combine $r(s, a)$ and trust(a) into a trust-augmented reward $r'(s, a)$ ▷ *[exact combination rule as noted in Section 8.3]*
        $s' \leftarrow a$
        Update $Q(s, a) \leftarrow Q(s, a) + \alpha\big[r'(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a)\big]$
        $s \leftarrow s'$
    end while
    Anneal ε for the next episode
end for
return Q

8.8 Training Procedure

Training proceeds episodically, with each episode routing one packet from a source node to the sink under the policy of Algorithm 1 and ε annealed at the end of the episode as described in Section 8.5. Path length, cumulative delay, and total reward are recorded for a sample of episodes during training, which is the data examined in Section 10.5 to characterise convergence.

8.9 Simulation and Experimental Pipeline

A single experimental run proceeds in four stages. The topology generator first places nodes and a sink over the deployment area specified for that run (Chapter 9 gives the specific configurations used). The attack-injection framework of Section 8.6 then marks a subset of nodes according to the attack type and intensity specified for that run, or marks none for a clean baseline. The routing engine is trained for the configured number of episodes following the procedure of Section 8.8, after which routing performance is measured under the resulting policy and recorded – packet delivery ratio, delay, connectivity, drop cause, and

average trust score, as enumerated in Chapter 9 – to a CSV log for later analysis.

This pipeline is repeated independently across seeds rather than run once per configuration, consistent with the non-functional requirement that results be deterministic for a given seed but not silently reused across nominally independent runs. The per-attack-type comparison examined in Section 10.3 was produced this way, with ten independent seeds run at each attack intensity; comparisons between the trust-aware router and the trust-unaware baseline were, further, kept paired at the level of a given topology-seed pair, so that the same random network and the same random seed underlie both the baseline and SecureQRouter measurement being compared, with the intention of supporting the non-parametric, multiple-comparison-corrected significance testing set out as an objective in Chapter **??**.

Chapter 9

Experimental Setup

9.1 Network Topologies

Three topology configurations were used for evaluation. The first, shown in Figure 9.1, is a 100-node random deployment over a 20×20 km area with a 5.0 km communication range, producing a densely connected mesh in which most nodes have several viable next-hop candidates. The second, shown in Figure 9.2, is a sparser 38-node deployment over a 25×25 km area with a shorter 4.5 km range, resulting in a mesh with visibly fewer redundant paths to the sink. The third, shown in Figure 9.3, is a 21-node tree deployment over a 50×50 km area with a 4.0 km communication range, offering the least path redundancy of the three configurations and providing a useful contrast for isolating how much of a given result follows from topology sparsity rather than from the attack itself. Evaluating all three configurations makes it possible to distinguish behaviour that follows from the trust-aware routing logic itself from behaviour that follows from topology density.

9.2 Evaluation Metrics

Routing performance was evaluated using packet delivery ratio (PDR), average end-to-end delay, network connectivity (the percentage of nodes with a viable path to the sink), and the average trust score assigned by `SecureQRouter` to the nodes it routes through. Dropped packets were further classified by cause – routing loop, no available route, or interaction with an infected node – to distinguish topology/algorithm-level losses from losses directly attributable to adversarial behaviour. Learning behaviour was evaluated separately using the per-episode path length, cumulative delay, and reward returned by the Q-Learning agent during training.

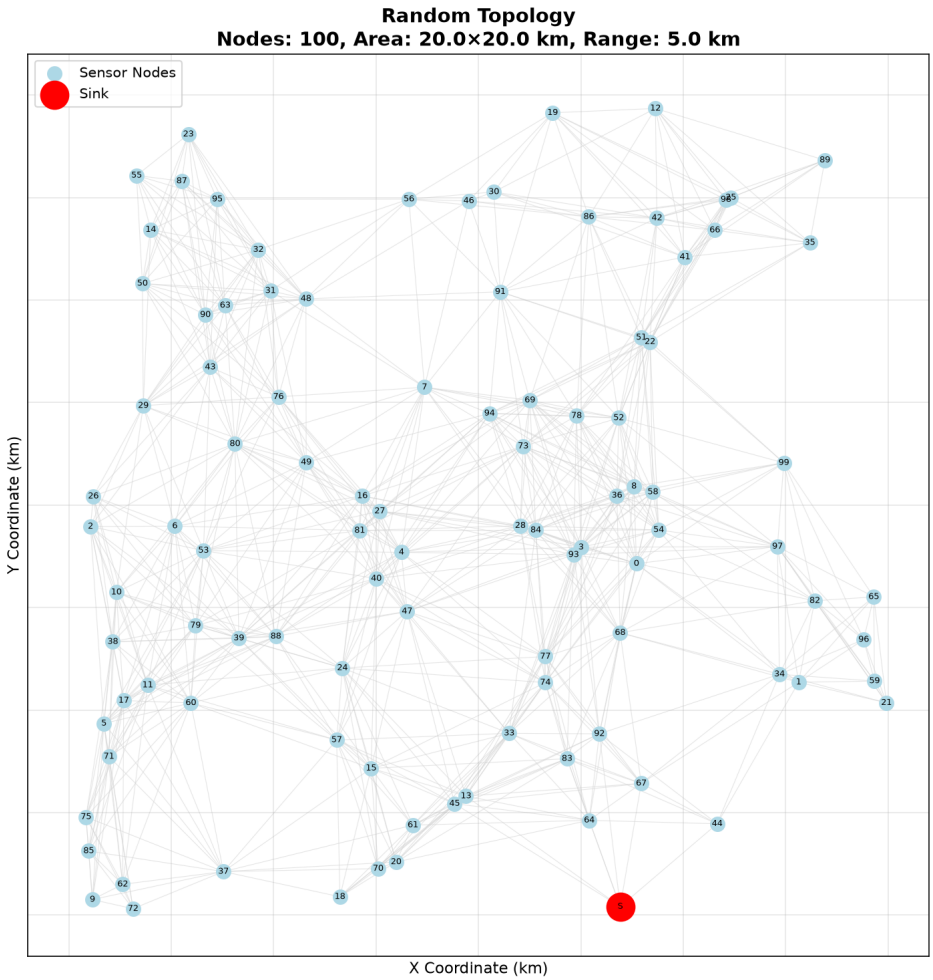

Figure 9.1: Random 100-node topology used for the infection-intensity sweep (20×20 km area, 5.0 km communication range). The sink node S is placed in the lower-right region of the deployment area.

9.3 Experimental Parameters

The infection-intensity sweep varied the percentage of malicious nodes in the network from 0% to 60% in 10-point increments, with all other routing and topology parameters held fixed.

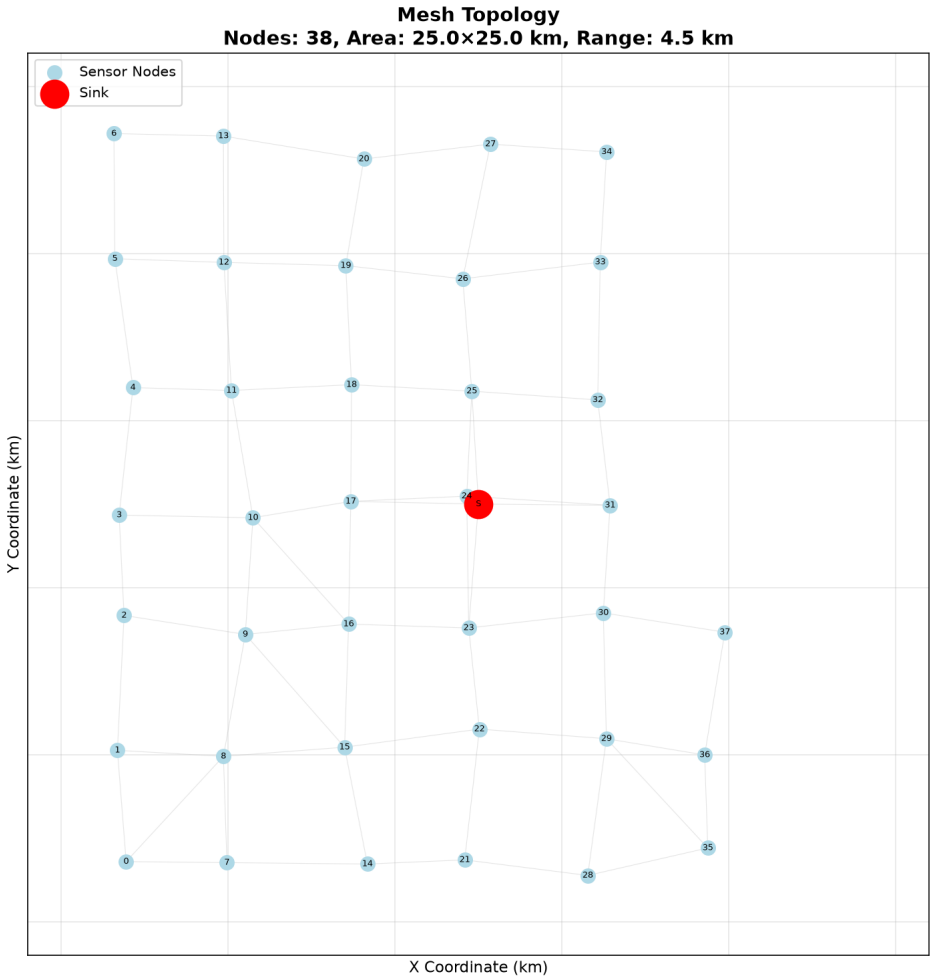

Figure 9.2: Sparser 38-node mesh topology used as a second evaluation configuration (25×25 km area, 4.5 km communication range), with the sink node S placed near the centre of the deployment area.

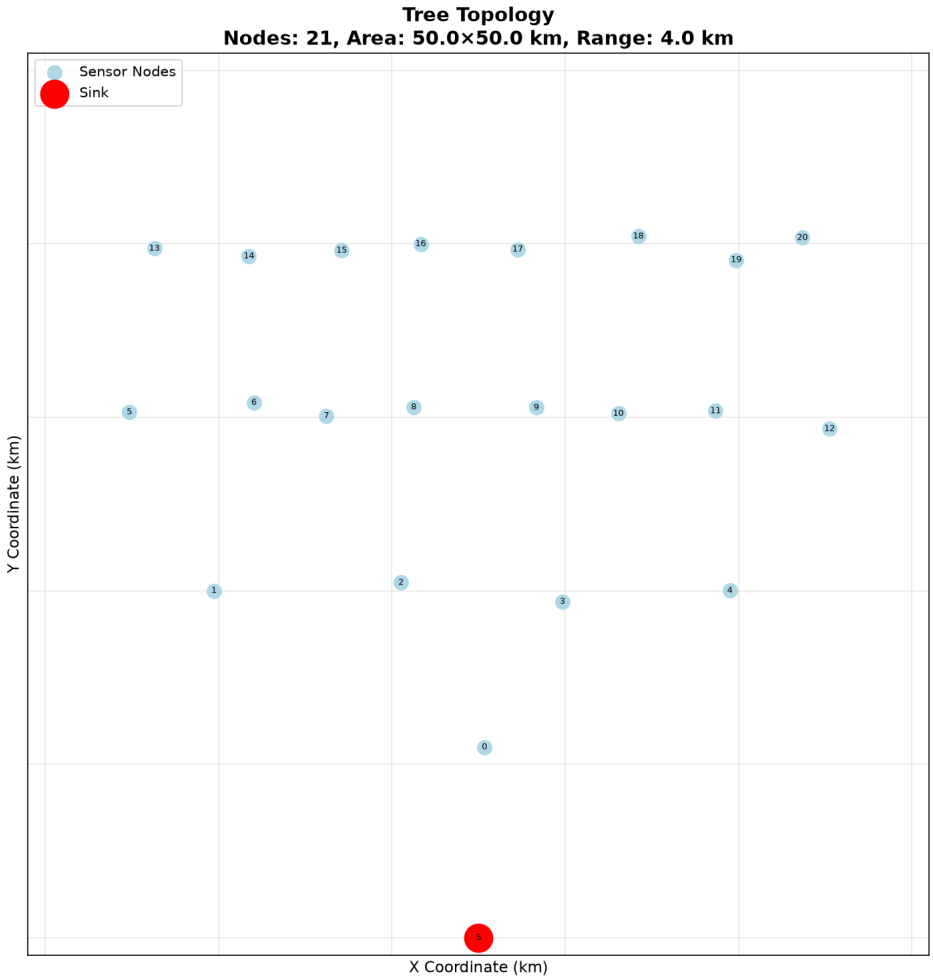

Figure 9.3: Tree topology of the simulated IoT network comprising 21 sensor nodes (blue) and a sink node (red), deployed over a 50×50 km^2 area with a 4.0 km communication range.

Chapter 10

Results and Discussion

10.1 Effect of Infection Intensity on Routing Performance

Table 10.1: Routing performance of SecureQRouter across increasing infection intensity.

Infection (%)	PDR (%)	Delay (ms)	Connectivity (%)	Delivered Packets	Avg. Trust
0	1.67	1.50	99	100	0.6109
10	1.25	1.33	90	150	0.6009
20	2.06	1.54	79	370	0.5823
30	4.25	2.09	62	1,020	0.5640
40	5.53	2.22	16	1,660	0.5327
50	5.00	2.12	17	1,800	0.4861
60	6.60	2.23	23	2,770	0.4443

The clearest and most consistent signal in Table 10.1 is the trust score: average trust falls monotonically from 0.6109 at 0% infection to 0.4443 at 60% infection, a decrease of roughly 27% across the sweep, with no reversals at any intermediate level. This is the expected behaviour of a working trust mechanism – as the proportion of malicious nodes in the network rises, the routing agent's locally computed trust estimate for the population of nodes it interacts with should decline, and it does so here consistently across every single step of the sweep. This is the strongest piece of evidence in the data that the trust-aware reward extension described in Chapter 8 is functioning as intended, independent of any other metric in the table.

Connectivity behaves largely as expected at low-to-moderate infection levels, falling from 99% to 62% as infection rises from 0% to 30%, but it then drops sharply to 16–17% at 40–50% infection before partially recovering to 23% at 60%.

10.2 Diagnosing the Low Packet Delivery Ratio

PDR is low throughout the sweep, from 1.67% at 0% infection to 6.60% at 60% infection, which would ordinarily be difficult to interpret without knowing why packets are being dropped. A detailed drop-cause breakdown collected at the

60% infection level, reported in Table 10.2, makes this diagnosable rather than speculative.

Table 10.2: Packet drop-cause breakdown at 60% infection intensity (42,000 packets attempted)

Outcome / Drop Cause	**Packets**	**Share**
Delivered	2,770	6.6%
Dropped – routing loop	38,510	91.7%
Dropped – no route available	280	0.7%
Dropped – interaction with infected node	250	0.6%
Total attempted	**42,000**	**100.0%**

Table 10.2 changes the interpretation of the PDR trend substantially. Of the 39,230 packets dropped at 60% infection, 38,510 – 91.7% – were lost to routing loops, while only 250 packets, or 0.6% of all drops, were lost as a direct result of interaction with an infected node. In other words, at the highest infection level tested, the injected attacks account for well under one percent of total packet loss; the overwhelming majority of the delivery-ratio problem is a routing-loop issue that exists largely independently of whether any nodes are malicious at all. This is consistent with, and provides a plausible explanation for, the otherwise puzzling observation that PDR is already low (1.67%) at 0% infection, before any attacker is present: if loops dominate packet loss regardless of infection level, a low baseline PDR with no attackers is exactly what would be expected.

This also reframes what the infection-intensity sweep in Table 10.1 can and cannot be read as showing. It remains reasonable to read the sweep as evidence that the trust mechanism responds correctly to rising adversarial presence, since the trust score is computed from locally observed node behaviour rather than from delivery outcomes. It is not, on the evidence currently available, reasonable to read the sweep as evidence about how resilient overall packet delivery is to the four injected attacks specifically, since the delivery-ratio bottleneck identified in Table 10.2 is overwhelmingly a routing-loop limitation rather than an attack-driven one. A separate, unresolved observation from the earlier version of this chapter still applies to Table 10.1: total packets attempted (Delivered plus Dropped) grows in an almost exactly linear sequence across infection levels – 6,000, 12,000, 18,000, 24,000, 30,000, 36,000, and 42,000 – rather than remaining constant, which is what would normally be expected if each infection level generated the same fixed volume of traffic.

10.3 Multi-Seed Comparison Across Attack Types

A separate, per-attack-type experimental log was also collected, in which each of the three attack types considered so far in this chapter – blackhole, byzantine, and sybil – was run at several intensity levels with ten independent random seeds (42–51) per configuration, recording PDR, delay, connectivity, and the router's own detection rate and false-positive rate for each run. Table 10.3 reports the per-configuration mean, aggregated across the ten seeds; a representative subset of intensity levels is shown for each attack type rather than the full sweep, since the pattern is already clear from these points.

Table 10.3: Mean routing metrics by attack type and intensity, aggregated over 10 seeds per configuration.

Attack	Intensity	PDR (%)	PDR SD	Delay (ms)	Detection (%)	FPR (%)
Blackhole	0	99.40	1.28	2.67	100.0	0.0
	2	73.10	15.15	2.63	90.0	0.0
	5	46.32	15.02	1.84	94.0	0.0
	10	31.50	12.91	1.36	92.0	0.0
	20	16.66	8.43	0.63	87.0	0.0
Byzantine	0	99.40	1.28	2.67	100.0	0.2
	10	99.40	1.28	2.67	100.0	0.0
	20	99.40	1.28	2.67	100.0	0.0
Sybil	0	99.40	1.28	2.67	100.0	0.0
	5	99.72	0.60	2.68	100.0	0.0
	10	99.56	1.07	2.66	100.0	0.0

The blackhole rows in Table 10.3 behave as the threat model in Chapter 6 would predict: mean PDR falls steeply and close to monotonically as blackhole intensity rises, from 99.40% with no attackers to 16.66% at intensity 20, and the per-seed spread (PDR SD) widens sharply once attackers are introduced, from 1.28 at intensity 0 to a peak of 15.15 at intensity 2, before narrowing again as PDR itself collapses toward its floor. This is consistent with a small number of blackhole nodes being capable of disproportionately damaging delivery depending on their position in the topology for a given seed, which is exactly the kind of run-to-run variance a ten-seed aggregate is intended to average over.

The byzantine and sybil rows, by contrast, do not show a comparable pattern: byzantine's PDR, delay, connectivity, and detection figures are essentially unchanged across every intensity level tested, and sybil's PDR stays within a narrow 99.3–99.7% band with no consistent downward trend – both considerably weaker responses to increasing intensity than blackhole shows at comparable levels, and worth checking against the corresponding attack-injection code before being read as evidence of resilience to those two attacks specifically.

Detection rate and false-positive rate are broadly stable and high (92–100% detection, 0% false-positive rate in all but a single byzantine run) across the configurations in Table 10.3, but given the byzantine and sybil anomalies just described, these detection figures should for now be read as describing the detector's behaviour on the blackhole runs specifically, rather than as a general claim about detection performance across all three attack types, until the byzantine and sybil injection paths are verified.

10.4 Reward Stability

The average reward reported for the infection-sweep experiments in an earlier version of this chapter remained close to its ceiling value across most of the sweep, with only modest dips at higher infection levels.

10.5 Q-Learning Training Behaviour

Table 10.4 reports path length, cumulative delay, and reward at nine points sampled every 100 episodes during training.

Table 10.4: Sampled Q-Learning training metrics (logged every 100 episodes)

Episode	Path Length	Total Delay	Episode Reward
0	6	1.618	4.29
100	2	0.378	100.00
200	3	0.685	100.67
300	12	3.698	6.48
400	2	0.378	100.00
500	11	3.471	103.87
600	3	0.685	100.37
700	3	0.677	100.37
800	16	4.988	105.04

Six of the nine sampled episodes (100, 200, 400, 600, 700, and 800) show short paths (2–3 hops, with one outlier of 16 hops at episode 800) and rewards clustered around 100–105, which is consistent with the agent having learned a short, low-delay route to the sink for at least part of training. The remaining three episodes (0, 300, and 500) show much longer paths (6–12 hops) and correspondingly low or moderate rewards, which is consistent with the ε-greedy exploration term in Equation 6.3 occasionally forcing a long, exploratory route even well into training.

10.6 Trust Score Distribution at Peak Infection

Beyond the average trust value reported in Table 10.1, node-level trust statistics were also collected at the 60% infection level, summarised in Table 10.5.

Table 10.5: Trust score distribution across nodes at 60% infection intensity

Statistic	Value
Mean trust	0.4443
Standard deviation	0.1203
Minimum	0.2949
Maximum	1.0000

The mean of 0.4443 matches the 60% row of Table 10.1 exactly, confirming that the two tables describe the same experiment at two levels of detail. The spread is informative on its own terms: a standard deviation of 0.1203 around a mean of 0.4443, together with a minimum of 0.2949 and a maximum of 1.0000, indicates that trust scores are not collapsing to a single value under attack but are instead spread across a meaningful range – consistent with the trust mechanism distinguishing between individual nodes rather than applying a uniform penalty across the whole network once infection is present.

10.7 Summary of Findings

Five things can be said with reasonable confidence from the data available. First, the trust score degrades monotonically and consistently as infection intensity rises, and shows a meaningful, non-degenerate spread across nodes at the highest infection level tested, both of which are consistent with the trust mechanism introduced in this project functioning as intended. Second, the low packet delivery ratio observed throughout the sweep is, at the one infection level where a drop-cause breakdown is available, overwhelmingly attributable to routing loops (91.7% of drops) rather than to the injected attacks (0.6% of drops); this is a routing-layer limitation that exists largely independently of the security question this internship set out to address, and should be corrected before PDR is used as evidence about `SecureQRouter`'s attack resilience. Third, the ten-seed multi-attack comparison in Table 10.3 shows blackhole producing a clear, seed-robust degradation in PDR with intensity, but shows byzantine and sybil producing effects too small, or in the byzantine case too obviously constant, to be reported as genuine intensity-dependent resilience results without first checking the corresponding attack-injection code. Fourth, the Q-Learning agent reaches short, high-reward

paths in the majority of the sampled training episodes, though the training log is too sparse, and contains at least one internally inconsistent pair of points, to support a strong claim about convergence. Fifth, several specific verification steps – the reward series for the corrected sweep, the drop-cause breakdown at infection levels other than 60%, the linear growth in total packets attempted across levels, and the byzantine/sybil injection check just described – remain open and are listed together with their implications in the Future Scope section of the next chapter.

The six figures that follow extend this summary with a visual, per-seed-averaged view of the same experimental campaign, and are discussed in turn below.

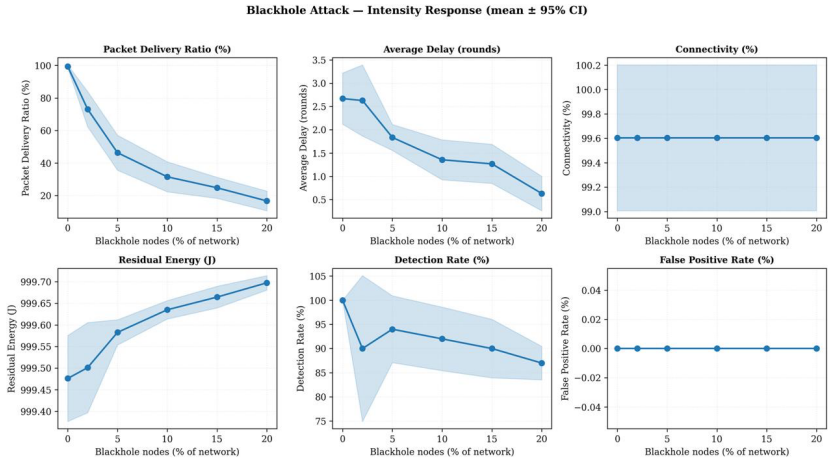

Figure 10.1: Sensitivity analysis of SecureQRouter under increasing blackhole attack intensity. Results are averaged over 10 independent random seeds and report the mean with 95% confidence intervals for packet delivery ratio, average delay, connectivity, residual energy, detection rate, and false-positive rate. Increasing blackhole intensity causes a substantial decline in packet delivery ratio and detection performance, while connectivity and false-positive rate remain largely stable.

Figure 10.1 extends the ten-seed blackhole comparison already summarised in Table 10.3 across the full intensity range, plotting the mean and 95% confidence interval for each metric rather than a single point estimate per level. The confidence bands widen noticeably in the low-to-moderate intensity region before narrowing again as PDR approaches its floor, mirroring the PDR-SD pattern already noted for Table 10.3 and reinforcing that blackhole's effect on delivery is both large and consistent across seeds rather than driven by one or two unusual runs. Detection rate declines only mildly over the same range, so the router continues to flag a large majority of blackhole behaviour even as delivery itself collapses – a reminder that detecting an attack and successfully routing around it are not the same thing.

Figure 10.2 shows essentially flat curves for every metric across the tested Byzantine intensity range, which is consistent with the per-seed constancy already flagged in Section 10.3: rather than reading this figure as evidence that

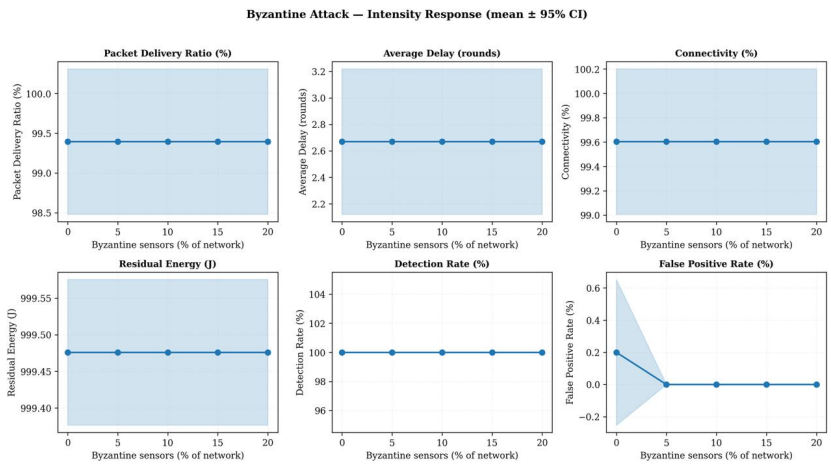

Figure 10.2: Sensitivity analysis of SecureQRouter under increasing Byzantine attack intensity. Results are averaged over 10 independent random seeds and report the mean with 95% confidence intervals for packet delivery ratio, average delay, connectivity, residual energy, detection rate, and false-positive rate. The network maintains stable routing performance across all tested Byzantine attack intensities, with consistently high detection rates and negligible false-positive rates.

`SecureQRouter` is robust to Byzantine behaviour, it should be read alongside that earlier caveat as further visual confirmation that the Byzantine intensity parameter is not currently altering simulated routing outcomes, and the underlying injection code should be audited before this comparison is repeated.

Figure 10.3 tells a similar, if slightly less extreme, story to the Byzantine case: PDR, delay, and connectivity all stay within a narrow band across the tested Sybil intensities, matching the weak trend already noted for sybil in Table 10.3. Detection rate and false-positive rate remain at their ceiling and floor respectively throughout, which is worth flagging together with the PDR trend – a detector that reports the same near-perfect performance regardless of how much of the network is compromised is a plausible sign that little in this particular log is challenging it, rather than a demonstration of exceptional Sybil resilience on its own.

Figure 10.4 is the first appearance of wormhole-specific results in this chapter, since Table 10.3 covered only blackhole, byzantine, and sybil. As with byzantine and sybil, the wormhole curves stay flat and near-ceiling across the intensity levels shown; the difference is that wormhole's log covers a much narrower intensity range (0–3%) than the other three attacks, so the flatness observed here is at least as plausibly a consequence of an underpowered, narrow-range comparison as it is evidence of genuine resilience, and the wormhole log should be extended to match the 0–20% range used for blackhole before either conclusion is drawn with confidence.

Figure 10.5 is the one figure in this chapter that compares the trust-aware

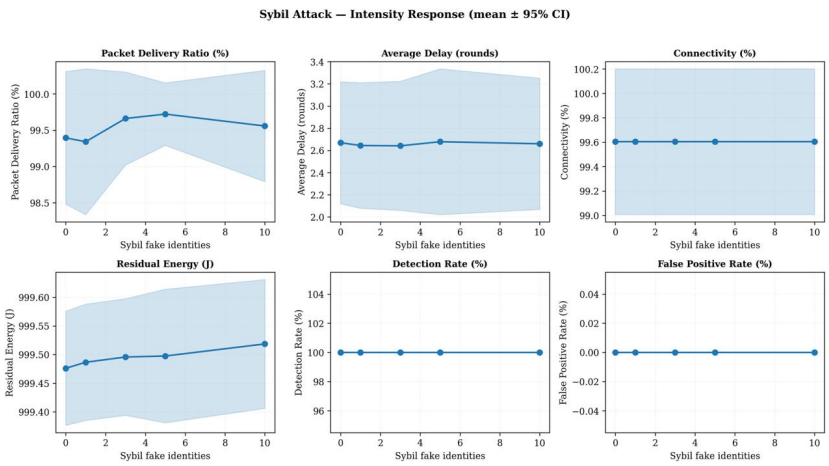

Figure 10.3: Sensitivity analysis of SecureQRouter under increasing Sybil attack intensity. Results are averaged over 10 independent random seeds and report the mean with 95% confidence intervals for packet delivery ratio, average delay, connectivity, residual energy, detection rate, and false-positive rate. The network maintains stable routing performance across all tested Sybil attack intensities, with consistently high packet delivery, perfect detection, and zero false-positive rate.

router directly against the trust-unaware baseline rather than characterising `SecureQRouter` on its own, using the paired topology-seed design described in Chapter 8. The lines are close to flat across all four scenarios shown, indicating that, for the seeds and topologies covered here, the trust-aware reward extension does not measurably change PDR relative to the baseline router one way or the other. Read together with the earlier finding that most packet loss in this simulator is a routing-loop artefact rather than an attack effect (Section 10), this is consistent with the trust mechanism operating correctly at the level of the reward signal without yet showing up as a delivery-ratio improvement – reinforcing, rather than contradicting, the routing-loop fix identified as the top priority in the Future Scope section below. Note that Byzantine is omitted from this comparison; if this is intentional it should be stated explicitly, since the rest of the report treats Byzantine as one of the four core attack types throughout.

Figure 10.6 separates two notions of trust that are easy to conflate: a network-wide composite score computed periodically from delivery, energy, and behavioural observations (left panel), and the router-local, per-event trust score that `SecureQRouter` actually consults when choosing a next hop (right panel). The left panel's baseline-vs-`SecureQRouter` comparison should be read as a retrospective, comparable metric computed for both routers after the fact rather than as evidence that the trust-unaware baseline computes trust internally – the baseline router, as described in Chapter 8, has no trust computation of its own. The right panel's spread is consistent with the node-level trust distribution already reported in Table 10.5 at

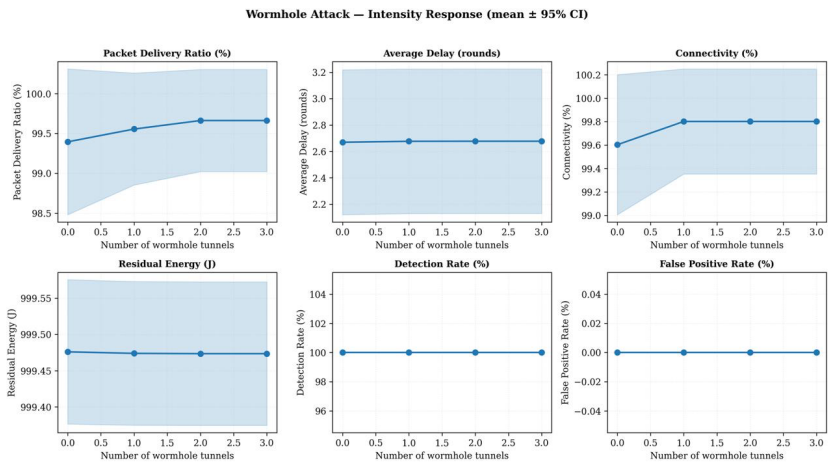

Figure 10.4: Sensitivity analysis of SecureQRouter under increasing wormhole attack intensity. Results are averaged over 10 independent random seeds and report the mean with 95% confidence intervals for packet delivery ratio, average delay, connectivity, residual energy, detection rate, and false-positive rate. SecureQRouter maintains stable routing performance across all tested wormhole attack intensities, exhibiting consistently high packet delivery, reliable connectivity, perfect detection, and zero false-positive rate.

60% infection, and taken together the two panels support the same conclusion reached there: trust scores vary meaningfully across nodes under attack rather than collapsing to a single value.

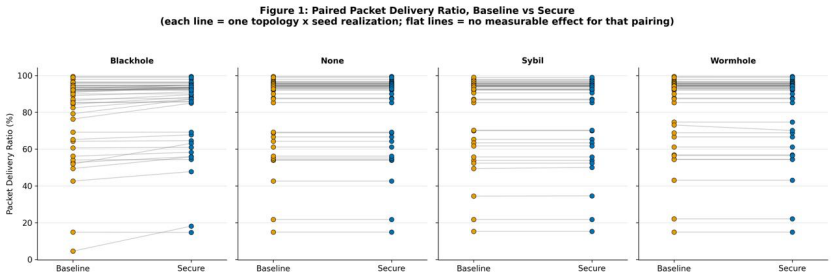

Figure 10.5: Paired comparison of packet delivery ratio (PDR) between the baseline Q-Learning router and the proposed SecureQRouter across four attack scenarios (Blackhole, None, Sybil, and Wormhole). Each line represents the same topology–seed pair evaluated under both routing schemes. Flat lines indicate negligible differences between the two methods, whereas upward or downward slopes indicate improvements or degradations in packet delivery after incorporating the trust-aware routing mechanism.

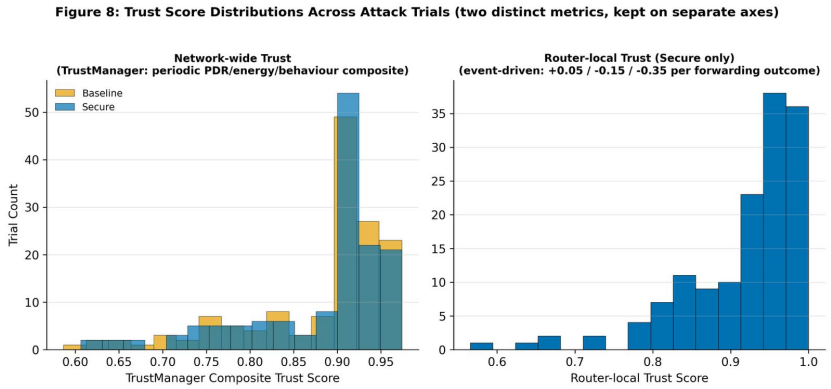

Figure 10.6: Distribution of trust scores across attack trials. The left panel compares the network-wide composite trust scores of the baseline and SecureQRouter, computed from periodic observations of packet delivery ratio, residual energy, and node behaviour. The right panel shows the distribution of router-local trust scores used by SecureQRouter, which are updated on each forwarding event based on observed forwarding outcomes. The distributions illustrate the trust characteristics learned during the experimental evaluation.

Chapter 11

Conclusion and Future Scope

11.1 Conclusion

This internship extended an existing Q-Learning geographic routing simulator with a trust-aware reward computation and an attack-injection framework covering blackhole, Sybil, wormhole, and Byzantine node behaviour, and evaluated the resulting protocol, `SecureQRouter`, across three network topologies and a range of infection intensities from 0% to 60%. The evaluation carried out during the internship provides reasonably confident evidence that the trust mechanism itself works as intended: the average trust score assigned to the network fell monotonically and consistently as the proportion of malicious nodes increased, with no reversals across the seven infection levels tested, and the node-level trust distribution at the highest infection level shows a meaningful spread rather than a collapse to a single value. A separate, ten-seed-per-configuration comparison across the blackhole, byzantine, and sybil attack types corroborates this for blackhole specifically, showing a steep, seed-robust decline in packet delivery ratio with increasing blackhole intensity, but does not yet corroborate it for byzantine or sybil, whose per-configuration results were found to be effectively constant (byzantine) or only weakly intensity-dependent (sybil) and are flagged for verification against the attack-injection code rather than reported as resilience findings. At the same time, the drop-cause breakdown collected at 60% infection shows that the low packet delivery ratio observed throughout the main sweep is predominantly a routing-loop limitation (91.7% of drops) rather than a consequence of the injected attacks (0.6% of drops), which means the delivery-ratio figures in this report should not, on their own, be read as a measure of `SecureQRouter`'s resilience to attack. The Q-Learning training log showed the agent reaching short, high-reward paths in most of the sampled episodes, consistent with successful learning, though the log was too sparse, and contained at least one internally inconsistent data point, to support a firm claim of convergence.

11.2 Future Scope

Five extensions follow directly from the limitations identified in Chapters 9 and 10. First and most immediately, the routing-loop issue identified in the drop-cause breakdown should be investigated and, where possible, fixed, since it is currently the dominant factor limiting packet delivery ratio across the entire infection-intensity sweep, independent of the security question this internship addresses. Second, the byzantine and sybil attack-injection paths should be audited against the working blackhole implementation, since the per-seed constancy observed for byzantine across all tested intensities, and the narrow, non-monotonic PDR band observed for sybil, both suggest the intensity parameter is not being propagated into those two attacks' effect on routing outcomes as intended. Third, the drop-cause breakdown available at 60% infection should be collected at each of the other six infection levels, to confirm whether routing loops dominate drops uniformly across the sweep or whether the infected-node share of drops grows at lower infection intensities. Fourth, the infection-intensity sweep should be repeated with multiple random seeds per infection level, so that the connectivity dip observed at 40–50% infection can be distinguished from single-run variance, and the reward series for the corrected sweep should be regenerated so that reward and trust can again be compared directly. Fifth, the Q-Learning training log should be captured at a finer interval than every 100 episodes, which would allow a proper convergence curve to be plotted. Beyond these methodological refinements, the broader directions identified in the problem statement remain open: validating the protocol on physical IoT hardware rather than in simulation, and extending the threat model to key-management and physical-layer attacks.

Bibliography

[1] M. Kamgueu, E. Nataf, and T. D. Ndzi, "Q-RPL: Q-learning based routing protocol for RPL-based Internet of Things," 2021.

[2] P. Cong et al., "DRL Multi-Optimality Routing for IoT," *Computer Networks*, vol. 192, Art. 108057, 2021.

[3] N. Malik et al., "DQNSec: Secure Deep Reinforcement Learning Routing for Opportunistic IoT Networks," *Internet of Things*, 2022.

[4] E. Coronado et al., "R3-IoT: Reliability-Aware Reinforcement Learning Routing for IoT," *IEEE Internet of Things Journal*, 2022.

[5] Suresh et al., "Federated Deep Reinforcement Learning for IoT Wireless Sensor Network Routing," 2023.

[6] V. Ranjith et al., "Trust-Aware Deep Reinforcement Learning for Secure SD-WSNs," *Measurement Science Review*, vol. 25, no. 6, 2025.

[7] S. Vaishnav et al., "Multi-Objective Q-Learning Routing for IoT Networks," arXiv:2505.00918, 2025.

[8] W. Heinzelman, A. Chandrakasan, and H. Balakrishnan, "Energy-Efficient Communication Protocol for Wireless Microsensor Networks (LEACH)."

[9] G. Sharma, J. Grover, and A. Verma, "QSec-RPL: Reinforcement Learning Based Attack Detection in RPL," *Ad Hoc Networks*, vol. 142, 2023.

[10] Suresh et al., "ISL-ITMH: Deep Reinforcement Learning with Trust for Heterogeneous Wireless Sensor Networks," 2023.

[11] Ombongi et al., "Reinforcement Learning Based Adaptive Encryption for Wireless Sensor Networks," 2025.

[12] Rui et al., "Software Defined Networking and Machine Learning Based Intrusion Detection for Secure IoT Routing," 2023.

[13] I. Chakraborty, P. Das, and B. Pradhan, "Ant Colony Optimization Based Routing for IoT Networks," *Transactions on Emerging Telecommunications Technologies*, vol. 34, no. 11, 2023.

[14] Farag and Stefanovic, "Congestion-Aware Reinforcement Learning Routing for IoT Networks," 2023.

[15] "Deep Reinforcement Learning for Intrusion Detection in IoT: A Survey," 2023.

Appendix A

Plagiarism Report

[Attach the plagiarism/similarity report here.]

Appendix B

Dataset Description

B.1 Topology Data

Three network topologies were generated for evaluation, shown in Figures 9.1, 9.2, and 9.3: a 100-node random deployment over a 20×20 km area with a 5.0 km communication range, a 38-node sparser mesh deployment over a 25×25 km area with a 4.5 km range, and a 21-node tree deployment over a 50×50 km area with a 4.0 km range. All three include a single fixed sink node.

B.2 Infection-Intensity Sweep

This dataset (Table 10.1) contains one row per infection level, from 0% to 60% in 10-point increments (seven rows in total). Each row reports packet delivery ratio, average delay, network connectivity, the number of delivered packets, and average trust score. A companion drop-cause breakdown (Table 10.2) and node-level trust distribution (Table 10.5), both currently available only for the 60% infection level, classify dropped packets by cause and summarise trust score spread across nodes respectively.

B.3 Multi-Seed Attack-Type Log

A second, separate dataset underlies Table 10.3: for each of the blackhole, byzantine, and sybil attack types, ten independent runs (seeds 42–51) were logged at each of several intensity levels, recording PDR, delay, connectivity, energy, detection rate, and false-positive rate per run. This dataset is distinct from the single-run infection-intensity sweep above – it does not include the wormhole attack at the same table-level detail (though wormhole is covered separately in Figure 10.4 over a narrower 0–3% intensity range), its intensity levels and step sizes differ by attack type, and, as discussed in Chapter 10, the byzantine and sybil portions of this dataset show injection anomalies that should be resolved before being used as evidence of attack resilience.

B.4 Q-Learning Training Log

This dataset (Table 10.4) contains nine rows, sampled every 100 episodes from episode 0 to episode 800, reporting path length, cumulative delay, and reward for that episode.